

Here is an analysis and structured overview of the **Implement Client Script & UI Policy (Incident)** project details provided:

Project Summary

- **Goal:** Enforce client-side data integrity, auto-populate values, control field visibility/read-only behavior, and prevent improper submissions or list-edits on ServiceNow Incident forms.
- **Core Skills:** Incident Management, UI Policy, UI Policy Actions, Client Scripts (onChange, onSubmit, onCellEdit), Form Validation.

Implementation Breakdown

1. UI Policy & Action Setup

- **UI Policy Name:** High Impact Control on the Incident table.
- **Condition:** Triggered when Impact is 1 - High.
- **Settings:** Active set to true, Reverse if false set to true (reverts rules when Impact changes away from High).
- **UI Policy Action:**
 - **Target Field:** Urgency
 - **Behavior:** Set Read-only to true when the condition is met.

2. Client Scripts

- **onChange Client Script:**
 - **Name:** Auto set urgency for high impact
 - **Table:** Incident | **Field:** impact | **Type:** onChange
 - **Logic:** Automatically sets urgency to 1 (High) and adds an informational message when impact is updated to 1.

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == "") {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

- **onSubmit Client Script:**
 - **Name:** Prevent save if Assigned To missing
 - **Table:** Incident | **Type:** onSubmit
 - **Logic:** Blocks form submission and shows an error box on assigned_to if impact is High (1) but assigned_to is empty.

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == "") {  
        g_form.showErrorBox(  

```

```

        'assigned_to',
        'Assigned To is mandatory for High impact incidents.'
    );
    return false;
}
return true;
}

```

- **onCellEdit Client Script:**

- **Name:** Prevent state change via list edit
- **Table:** Incident | **Field:** state | **Type:** onCellEdit
- **Logic:** Blocks users from editing the State field directly from list view and alerts them to use the Incident form instead.

```

function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```

    callback(false);
}

```

Testing & Verification Matrix

Test Activity	Test Condition	Expected Result	Pass/Fail Criteria
Activity 1: Mandatory Validation	Set Impact = High, leave Assigned To empty, submit form.	Form submission is blocked; error appears under Assigned To.	Save blocked with error message.
Activity 2: Successful Save	Set Impact = High, populate Assigned To, submit form.	Urgency auto-sets to 1, becomes read-only, and record saves.	Form submits cleanly.
Activity 3: Reverse Condition	Change Impact from High to Medium.	Urgency becomes editable again; Assigned To constraint relaxes.	Fields revert as configured.
Activity 4: List Edit Block	Double-click State field in Incident list view.	Alert displays; state change is rejected via callback(false).	List edit denied.
Activity 5: Form-Based State Update	Open Incident form and change State.	Form updates successfully.	State change accepted via form.